



JAVA **STREAMS**

101



What are Java Streams?

- A sequence of elements supporting **sequential and parallel operations**
- Process data declaratively
- **Chain operations** elegantly
- Write cleaner, more **readable code**



Creating Streams

Code Snippet:

```
// From Collection
List<String> list = Arrays.asList("a", "b", "c");
Stream<String> stream = list.stream();

// From Array
String[] arr = {"a", "b", "c"};
Stream<String> stream = Arrays.stream(arr);

// Direct Creation
Stream<Integer> stream = Stream.of(1, 2, 3);
```



Filter Operation

Code Snippet:

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);  
List<Integer> evenNumbers = numbers.stream()  
    .filter(n -> n % 2 == 0)  
    .collect(Collectors.toList());  
// Result: [2, 4, 6]
```



Map Operation

Code Snippet:

```
List<String> names = Arrays.asList("alia", "katrina", "deepika");  
List<String> upperNames = names.stream()  
    .map(String::toUpperCase)  
    .collect(Collectors.toList());  
// Result: [ALIA, KATRINA, DEEPIKA]
```



Collect Operation

Code Snippet:

```
● ● ●  
  
// To List  
.collect(Collectors.toList())  
  
// To Set  
.collect(Collectors.toSet())  
  
// To Map  
.collect(Collectors.toMap(  
    Person::getName,  
    Person::getAge  
))
```



Reduce Operation

Code Snippet:

```
List<Integer> numbers =  
Arrays.asList(1, 2, 3, 4, 5);  
int sum = numbers.stream()  
    .reduce(0, (a, b) -> a + b);  
// Result: 15
```



Real-World Applications

Data Filtering

Data Transformation

Grouping and Aggregation

Statistical Calculations

Parallel Processing



Stream Best Practices

- Use streams for **declarative programming**
- Use **parallel streams** for large dataset
- Always **close streams** when using I/O operations



Performance Tips

- 1 **Filter before map** when possible
- 2 Use **parallel streams** carefully
- 3 **Avoid stateful operations** in parallel streams
- 4 **Don't overuse** streams for simple operation